

Data Pipeline Specification (EN)

Module: data_pipeline

Language: EN

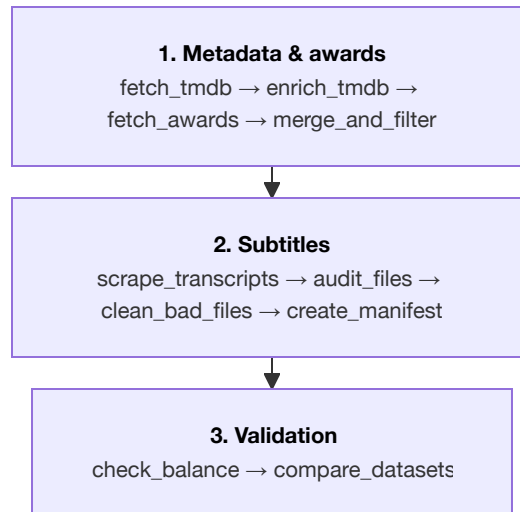
Table of Contents

- [1. Pipeline overview](#)
- [2. Scripts](#)
 - [2.1 fetch_tmdb.py](#)
 - [2.2 enrich_tmdb_data.py](#)
 - [2.3 fetch_awards.py](#)
 - [2.4 merge_and_filter.py](#)
 - [2.5 scrape_transcripts.py](#)
 - [2.6 audit_files.py](#)
 - [2.7 clean_bad_files.py](#)
 - [2.8 create_downloaded_manifest.py](#)
 - [2.9 check_balance.py](#)
 - [2.10 compare_datasets.py](#)

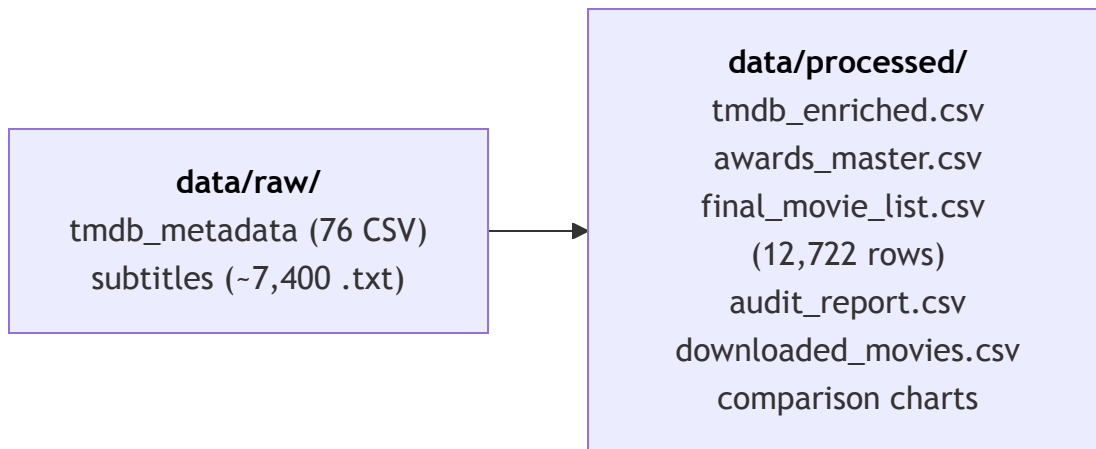
This module contains 10 scripts that run sequentially to build the complete movie dataset from scratch. Each script handles one stage — from fetching raw API data to producing a clean, analysis-ready dataset.

1. Pipeline overview

Sequential flow (10 scripts):



Data artifacts produced by the pipeline:



2. Scripts

2.1 `fetch_tmdb.py` — Fetch Movie Metadata

Builds the foundation of the dataset by collecting the top 500 movies per year from the TMDb API (1950–2025). Saves yearly CSV checkpoints so data is preserved if the process is interrupted.

Input: TMDb API key (`.env`)

Output: `data/raw/tmdb_metadata/movies_YYYY.csv` (76 files), `data/processed/tmdb_all_raw.csv` (consolidated)

2.2 `enrich_tmdb_data.py` — Add Financial Details

Fetches concrete financial metrics (budget, revenue) needed to compute ROI (the commercial success target). Also retrieves IMDb IDs for cross-referencing. Handles batch processing with crash recovery and automatically pauses when hitting API rate limits.

Input: `data/processed/tmdb_all_raw.csv`

Output: `data/processed/tmdb_enriched.csv`

2.3 `fetch_awards.py` — Collect Award Data

Produces the ground truth labels for the critical success prediction model. Uses the Wikipedia API to collect data on major film awards (Oscar, BAFTA, Golden Globe, etc.). Parses messy HTML tables, cleans titles, and consolidates everything into a single CSV.

Input: Wikipedia API

Output: `data/processed/awards_master.csv`

2.4 `merge_and_filter.py` — Merge and Filter

Acts as the dataset's quality gate. Merges TMDb metadata, financial data, and award records into one master table. Only keeps movies that pass at least one of three criteria: valid financial data, a

major award, or a high vote count (evidence of being a real, notable film).

Input: data/processed/tmdb_all_raw.csv , data/processed/tmdb_enriched.csv ,
data/processed/awards_master.csv

Output: data/processed/final_movie_list.csv (12,722 movies)

2.5 `scrape_transcripts.py` — Download Subtitle Texts

The main tool for obtaining the raw text that the NLP models will process. Spoofs a real web browser (rotating User-Agent headers) to bypass bot protection and scrape subtitles from the web. Uses fuzzy string matching to ensure each downloaded subtitle actually corresponds to the correct movie in the database.

Input: `data/processed/final_movie_list.csv`

Output: `data/raw/subtitles/{tmdb_id}.txt` (~7,400 files)

2.6 `audit_files.py` — Audit Subtitle Quality

Quality-checks all downloaded subtitle files before they enter the ML pipeline. Scans the entire subtitle folder looking for hidden corruption: files that are suspiciously small (<2 KB), contain HTML error pages ("404 Not Found"), or Cloudflare block pages.

Input: `data/raw/subtitles/`

Output: `data/processed/audit_report.csv`

2.7 `clean_bad_files.py` — Remove Corrupt Files

Deletes the corrupt files flagged by the audit script so they don't contaminate NLP feature extraction. Reads the audit report and removes bad files from disk. Has a built-in safety check that requires typing "DELETE" in the terminal before any permanent deletion.

Input: `data/processed/audit_report.csv`

Output: Corrupt files removed from `data/raw/subtitles/`

2.8 `create_downloaded_manifest.py` — Build Final Manifest

After cleanup, verifies what usable data actually remains. Cross-references the subtitle files physically present on disk against the filtered master movie list. Produces the definitive "source of truth" CSV manifest, guaranteeing every row in the training dataset has a matching valid `.txt` file.

Input: data/raw/subtitles/ , data/processed/final_movie_list.csv

Output: data/processed/downloaded_movies.csv

2.9 check_balance.py — Check Class Balance

Validates that the final dataset is statistically ready for machine learning. Analyzes class imbalance across target variables (e.g., the ratio of award winners to non-winners). Serves as early diagnostics — flags whether SMOTE or other resampling will be needed before training.

Input: data/processed/final_movie_list.csv , data/processed/audit_report.csv

Output: Console report with class distribution stats

2.10 compare_datasets.py — Analyze Collection Bias

Visually checks whether the web scraping process introduced any systematic bias into the data. Compares the movies originally targeted for download against those that were actually collected. Generates and saves pie and bar charts (ready for project documentation) showing whether the scraper struggled more with older movies or specific genres.

Input: data/processed/final_movie_list.csv , data/processed/downloaded_movies.csv

Output: Comparison charts (saved to reports/figures/)